

重庆大学计算机专业核心课程《机器学习》

第7章 贝叶斯分类器

授课教师：郑林江

第7章 贝叶斯分类器

➡ 7.1 贝叶斯决策论

7.2 极大似然估计

7.3 朴素贝叶斯分类器

7.4 半朴素贝叶斯分类器

7.5 贝叶斯网

7.6 EM算法

- 朴素贝叶斯算法是有监督的学习算法，解决的是**分类问题**。如客户是否流失、是否值得投资、信用等级评定等多分类问题
- **优点**: 在于简单易懂、学习效率高、在某些领域（垃圾邮件、文本分类）的分类问题中能够与决策树、神经网络相媲美。
- **缺点**: 由于该算法以自变量之间的独立（条件特征独立）性和连续变量的正态性假设为前提，就会导致算法精度在某种程度上受影响。

1、场景

垃圾消息识别



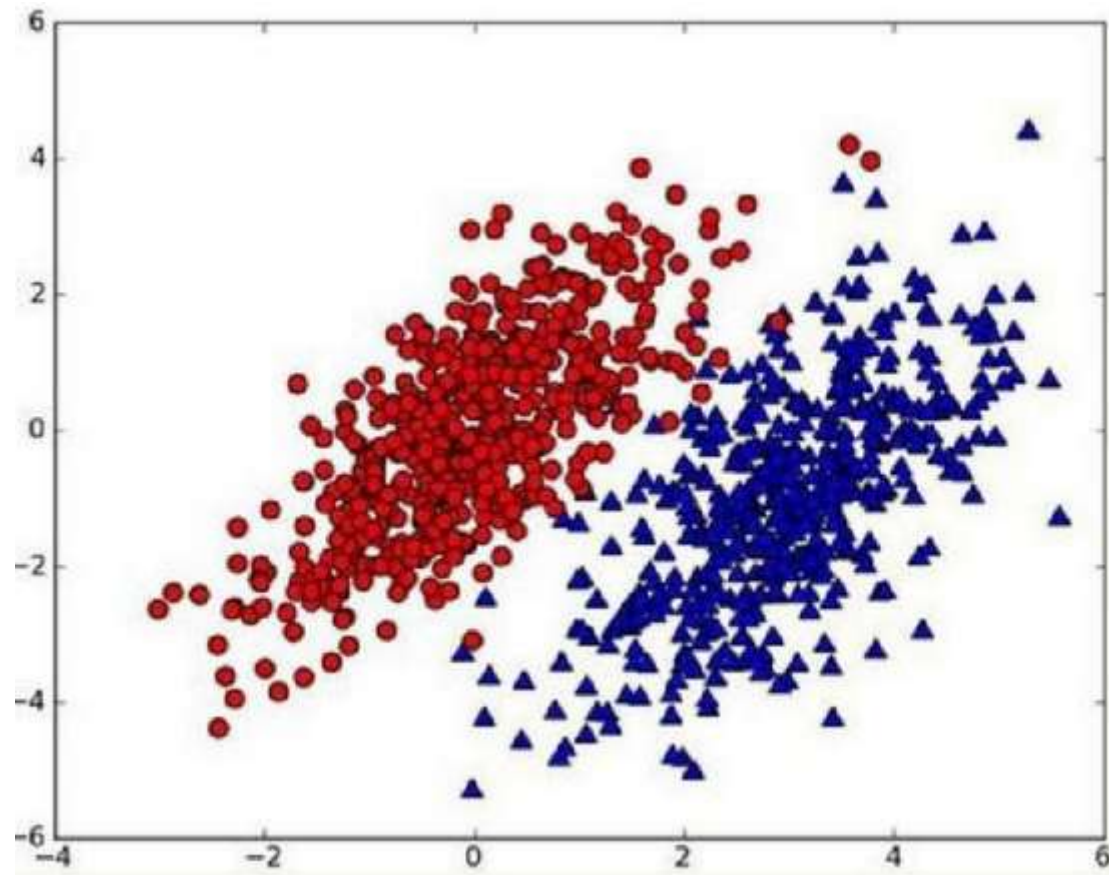
GIS+数据挖掘应用



2、基本原理

□ 假设现有一个数据集，由两类数据组成：

- ✓ 现在用 $p_1(x,y)$ 表示数据点 (x,y) 属于类别1(图中红色圆点表示的类别)的概率，
- ✓ 用 $p_2(x,y)$ 表示数据点 (x,y) 属于类别2(图中蓝色三角形表示的类别)的概率。



2、基本原理

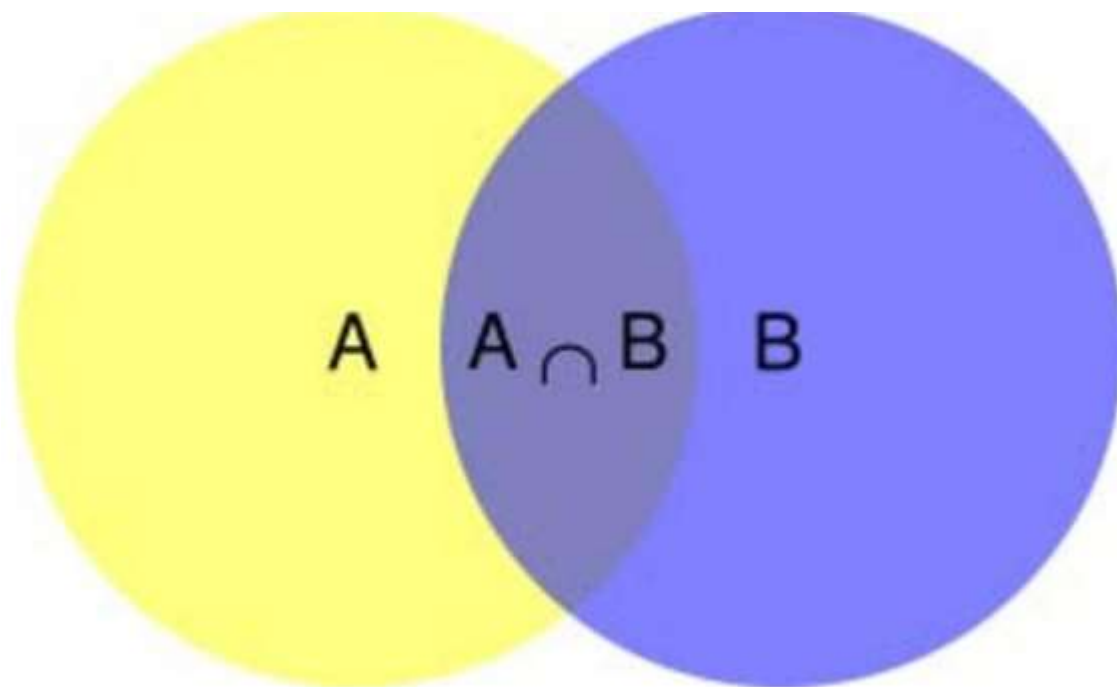
- 对于一个新数据点 (x,y) ，可以用下面的规则来判断它的类别：
 - 如果 $p_1(x,y) > p_2(x,y)$ ，那么类别为1
 - 如果 $p_1(x,y) < p_2(x,y)$ ，那么类别为2

- 也就是说，**会选择高概率对应的类别**。这就是贝叶斯决策理论的核心思想，即选择具有最高概率的决策。接下来，**就是学习如何计算 p_1 和 p_2 概率**。

3、条件概率

- **条件概率**(Conditional probability), 就是指在事件B发生的情况下, 事件A发生的概率, 用 $P(A|B)$ 来表示。
- 在事件B发生的情况下, 事件A发生的概率就是 $P(A \cap B)$ 除以 $P(B)$ 。

$$P(A|B) = P(A \cap B) / P(B)$$



3、条件概率

□ 条件概率的计算公式

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

$$P(A \cap B) = P(A|B)P(B)$$

$$P(A \cap B) = P(B|A)P(A)$$

$$P(A|B)P(B) = P(B|A)P(A)$$

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

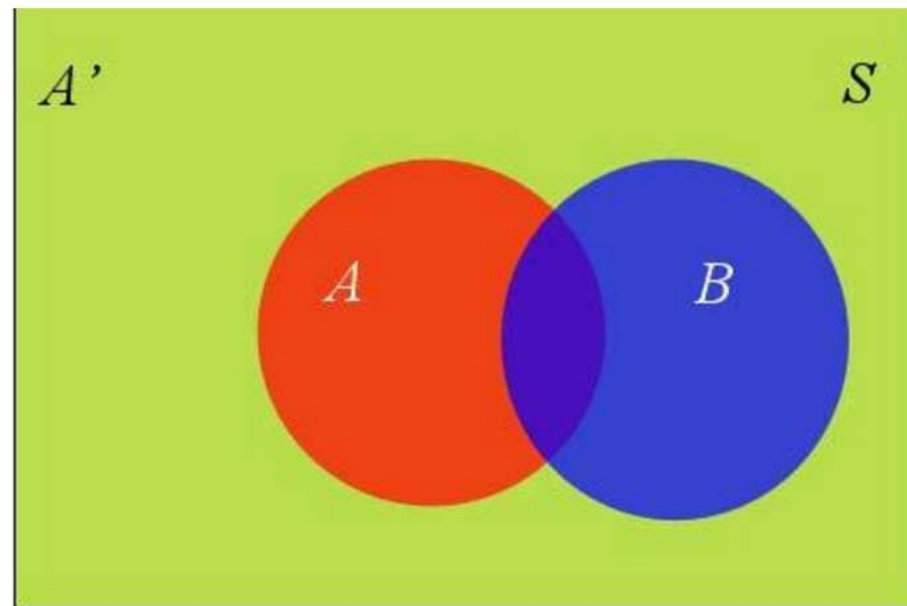
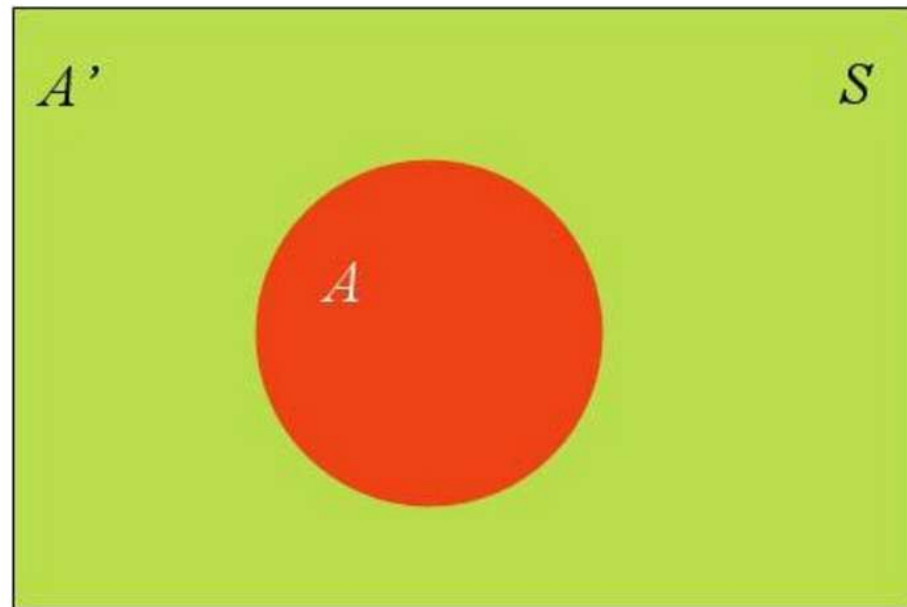
先验概率

后验概率

4、全概率公式

- 假定样本空间 S ，是两个事件 A 与 A' 的和。
- 红色部分是事件 A ，绿色部分是事件 A' ，它们共同构成了样本空间 S 。
- 在这种情况下，事件 B 可以划分成两个部分。

$$P(B) = P(B \cap A) + P(B \cap A')$$



4、全概率公式

□ 继续推导：

$$P(B) = P(B \cap A) + P(B \cap A')$$

□ 根据条件概率公式，已知：

$$P(B \cap A) = P(B|A)P(A)$$

□ 所以：

$$P(B) = P(B|A)P(A) + P(B|A')P(A')$$

□ 它的含义是，如果A和A'构成样本空间的一个划分，那么事件B的概率，就等于A和A'的概率分别乘以B对这两个事件的条件概率之和

4、全概率公式

□ 将：

$$P(B) = P(B|A)P(A) + P(B|A')P(A')$$

□ 带入全概率公式

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

□ 得到：

$$P(A|B) = \frac{P(B|A)P(A)}{P(B|A)P(A) + P(B|A')P(A')}$$

5、贝叶斯推断

- 对条件概率公式进行变形，可以得到如下形式：

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} \quad \Rightarrow \quad P(A|B) = P(A) \frac{P(B|A)}{P(B)}$$

- $P(A)$ 称为"**先验概率**" (Prior probability) ，即在B事件发生之前，对A事件概率的一个判断。
- $P(A|B)$ 称为"**后验概率**" (Posterior probability) ，即在B事件发生之后，对A事件概率的重新评估。
- $P(B|A)/P(B)$ 称为"**可能性函数**" (Likelyhood) ，这是一个调整因子，使得预估概率更接近真实概率

5、贝叶斯推断

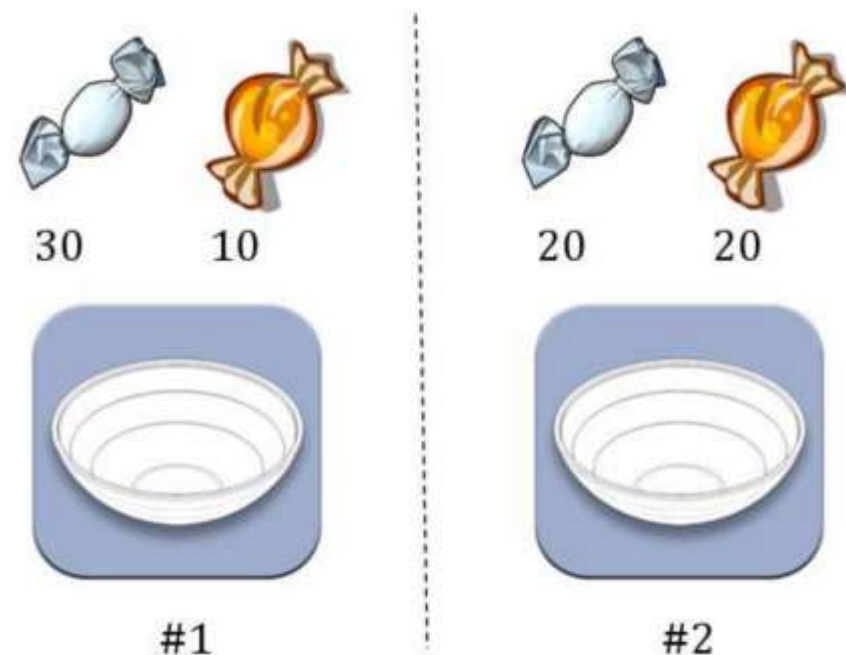
- 条件概率可以理解成下面的式子

$$\text{后验概率} = \text{先验概率} * \text{调整因子}$$

- 这就是贝叶斯推断的含义：先预估一个"先验概率"，然后加入实验结果，看这个实验到底是增强还是削弱了"先验概率"，由此得到更接近事实的"后验概率"。
- 如果"可能性函数" $P(B|A)/P(B) > 1$ ，意味着"先验概率"被增强，事件A的發生的可能性变大；如果"可能性函数" $= 1$ ，意味着B事件无助于判断事件A的可能性；如果"可能性函数" < 1 ，意味着"先验概率"被削弱，事件A的可能性变小。

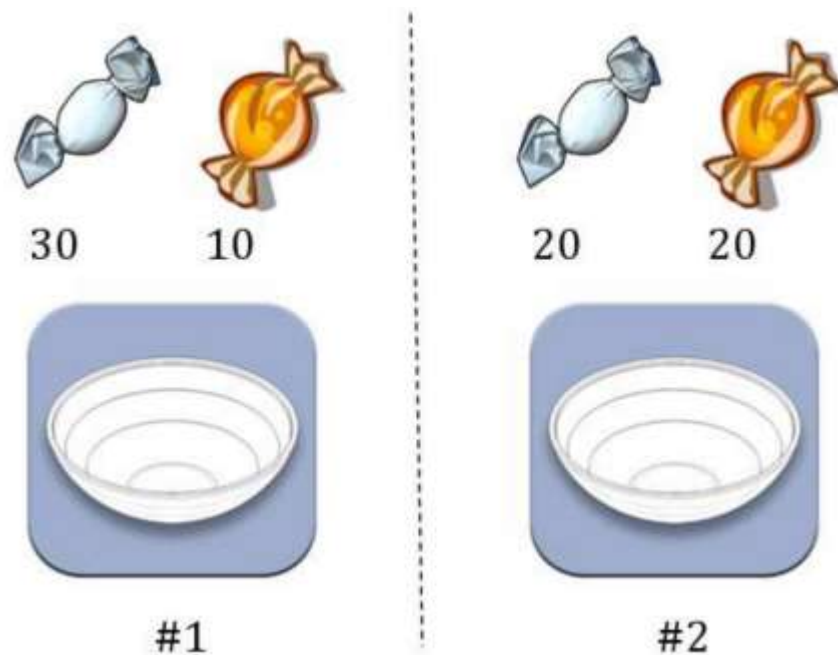
5、贝叶斯推断

- ◆ 两个一模一样的碗，一号碗有30颗水果糖和10颗巧克力糖，二号碗有水果糖和巧克力糖各20颗。现在随机选择一个碗，从中摸出一颗糖，发现是水果糖。请问这颗水果糖来自一号碗的概率有多大？
- ◆ 假定， H_1 表示一号碗， H_2 表示二号碗。由于这两个碗是一样的，所以 $P(H_1)=P(H_2)$ ，也就是说，在取出水果糖之前，这两个碗被选中的概率相同。因此， $P(H_1)=0.5$ ，这个概率就叫做“先验概率”，即没有做实验之前，来自一号碗的概率是0.5。



5、贝叶斯推断

- 再假定，E表示水果糖，所以问题就变成了在已知E的情况下，来自一号碗的概率有多大，即求 $P(H_1|E)$ 。这个概率叫做“**后验概率**”，即在E事件发生之后，对 $P(H_1)$ 的修正。
- 根据条件概率公式，得到
- 已知， $P(H_1)$ 等于0.5， $P(E|H_1)$ 为一号碗中取出水果糖的概率，等于 $30 \div (30 + 10) = 0.75$ ，那么**求出 $P(E)$** 就可以得到答案。



$$P(H_1|E) = P(H_1) \frac{P(E|H_1)}{P(E)}$$

5、贝叶斯推断

- 根据全概率公式：

$$P(E) = P(E|H_1)P(H_1) + P(E|H_2)P(H_2)$$

- 所以

$$P(E) = 0.75 \times 0.5 + 0.5 \times 0.5 = 0.625$$

- 代入原方程求得：

$$P(H_1|E) = 0.5 \times \frac{0.75}{0.625} = 0.6$$

- 即来自一号碗的概率是0.6。也就是说取出水果糖之后H1事件的可能性得到了增强。

- Q：为什么呢？

5、贝叶斯推断

- 新问题：现在随机选择一个碗，从中摸出一颗糖，发现是水果糖。请问这颗水果糖是来自一号碗还是二号碗？ Q：跟原来的问题有什么区别？
- 本质：不需要知道具体的类别概率，只需要知道所属类别。
- 是否有必要计算 $P(E)$ 这个全概率呢？其实只需要比较 $P(H1|E)$ 和 $P(H2|E)$ 的大小，找到那个最大的概率就可以。因为两者的分母都是相同的($P(E)$)，那只需要比较分子即可。即比较 $P(E|H1)P(H1)$ 和 $P(E|H2)P(H2)$ 的大小
- 所以为了减少计算量，全概率公式在实际编程中可以根据情况不使用

6、朴素贝叶斯推断

- 理解了贝叶斯推断，继续看看朴素贝叶斯。贝叶斯和朴素贝叶斯的概念是不同的，区别就在于“**朴素**”二字，朴素贝叶斯**对条件概率分布做了条件独立性的假设**。比如下面的公式，假设X有n个特征：

$$P(a|X) = p(X|a)p(a) = p(x_1, x_2, x_3, \dots, x_n|a)p(a)$$

- 由于每个特征都是独立的**，可以进一步拆分公式

$$\begin{aligned} p(a|X) &= p(X|a)p(a) \\ &= \{p(x_1|a) * p(x_2|a) * p(x_3|a) * \dots * p(x_n|a)\}p(a) \end{aligned}$$

7、举例

- 某个医院早上来了六个门诊的病人，他们的情况如下表所示：

症状	职业	疾病
打喷嚏	护士	感冒
打喷嚏	农夫	过敏
头痛	建筑工人	脑震荡
头痛	建筑工人	感冒
打喷嚏	教师	感冒
头痛	教师	脑震荡

7、举例

□ 现在又来了第七个病人，是一个打喷嚏的建筑工人。请问他患上感冒的概率有多大？

□ 根据贝叶斯定理可得：

$$P(\text{感冒}|\text{打喷嚏} \times \text{建筑工人}) = \frac{P(\text{打喷嚏} \times \text{建筑工人}|\text{感冒}) \times P(\text{感冒})}{P(\text{打喷嚏} \times \text{建筑工人})}$$

□ 根据朴素贝叶斯条件独立性的假设可知，"打喷嚏"和"建筑工人"这两个特征是独立的，因此，上面的等式就变成了

$$P(\text{感冒}|\text{打喷嚏} \times \text{建筑工人}) = \frac{P(\text{打喷嚏}|\text{感冒}) \times P(\text{建筑工人}|\text{感冒}) \times P(\text{感冒})}{P(\text{打喷嚏}) \times P(\text{建筑工人})}$$

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

7、举例

□ 计算得到

$$P(\text{感冒}|\text{打喷嚏} \times \text{建筑工人}) = \frac{0.66 \times 0.33 \times 0.5}{0.5 \times 0.33} = 0.66$$

- 因此，这个打喷嚏的建筑工人，有66%的概率是得了感冒。同理，可以计算这个病人患上过敏或脑震荡的概率。比较这几个概率，就可以知道他最可能得什么病。
- 这就是贝叶斯分类器的基本方法：在统计资料的基础上，依据某些特征，计算各个类别的概率，从而实现分类。
- Q: 直观上看，判断为感冒的概率为多少？如果原来的人全是感冒，会如何？

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- 贝叶斯分类器的原理是通过对象的先验概率，利用贝叶斯公式计算出其后验概率，即该对象属于某一类的概率，**选择具有最大后验概率的类作为该对象所属的类。**

- 主要特点：
 - 属性可以离散或连续
 - 数学基础扎实，分类效率稳定
 - 对缺失和噪声不太敏感
 - 属性如果不相关，分类效果较好

9、贝叶斯决策论

□ 贝叶斯决策论 (Bayesian decision theory) 是在概率框架下实施决策的基本方法。

✓ 在分类问题情况下，在所有相关概率都已知的理想情形下，贝叶斯决策考虑如何基于这些概率和误判损失来选择最优的类别标记。

✓ 假设有 N 种可能的类别标记，即 $y = \{c_1, c_2, \dots, c_N\}$ ， λ_{ij} 是将一个真实标记为 c_j 的样本误分类为 c_i 所产生的损失。基于后验概率 $P\{c_i | \mathbf{x}\}$ 可获得将样本 $\mathbf{x}$ 分类为 c_i 所产生的期望损失 (expected loss)，即在样本 $\mathbf{x}$ 上的“条件风险” (conditional risk)

$$R(c_i | \mathbf{x}) = \sum_{j=1}^N \lambda_{ij} P(c_j | \mathbf{x}) \quad (7.1)$$

✓ 我们的任务是寻找一个判定准则 $h: X \mapsto Y$ 以最小化总体风险

$$R(h) = \mathbf{E}_x [R(h(\mathbf{x}) | \mathbf{x})] \quad (7.2)$$

9、贝叶斯决策论

□ 显然，对每个样本 x ，若 h 能最小化条件风险 $R(h(\mathbf{x}) | \mathbf{x})$ ，则总体风险 $R(h)$ 也将被最小化。

✓ 这就产生了 **贝叶斯判定准则** (Bayes decision rule)：为最小化总体风险，只需在每个样本上选择那个能使条件风险 $R(c | \mathbf{x})$ 最小的类别标记，即

$$h^*(x) = \underset{c \in y}{\operatorname{argmin}} R(c | x) \quad (7.3)$$

✓ 此时， $h^*(x)$ 被称为贝叶斯最优分类器 (Bayes optimal classifier)，与之对应的总体风险 $R(h^*)$ 称为贝叶斯风险 (Bayes risk)

✓ $1 - R(h^*)$ 反映了分类起 **所能达到的最好性能**，即通过机器学习所能产生的模型精度的理论上限。

9、贝叶斯决策论

□ 具体来说，若目标是最小化分类错误率，则误判损失 λ_{ij} 可写为

$$\lambda_{i,j} \begin{cases} 0, & \text{if } i = j; \\ 1, & \text{otherwise,} \end{cases} \quad (7.4)$$

□ 此时条件风险

$$R(c | \mathbf{x}) = 1 - P(c | \mathbf{x}) \quad (7.5)$$

□ 于是，最小化分类错误率的贝叶斯最优分类器为

$$h^*(x) = \operatorname{argmax}_{c \in y} P(c | x) \quad (7.6)$$

- 即对每个样本 $\mathbf{x}$ ，选择能使后验概率 $P(c | \mathbf{x})$ 最大的类别标记。

9、贝叶斯决策论

- 不难看出，使用贝叶斯判定准则来最小化决策风险，首先**要获得**后验概率 $P(c | \mathbf{x})$ 。
- 然而，在现实任务中通常难以直接获得。机器学习所要实现的是基于有限的训练样本尽可能准确地**估计出**后验概率 $P(c | \mathbf{x})$ 。
- 主要有两种策略：
 - **判别式模型** (discriminative models)
 - 给定 $\mathbf{x}$ ，通过直接建模 $P(c | \mathbf{x})$ ，来预测 c
 - 决策树，BP神经网络，支持向量机
 - **生成式模型** (generative models)
 - 先对联合概率分布 $P(\mathbf{x}, c)$ 建模，再由此获得 $P(c | \mathbf{x})$
 - 生成式模型考虑

$$P(c | \mathbf{x}) = \frac{P(\mathbf{x}, c)}{P(\mathbf{x})} \quad (7.7)$$

9、贝叶斯决策论

□ 生成式模型

$$P(c | \mathbf{x}) = \frac{P(\mathbf{x}, c)}{P(\mathbf{x})} \quad (7.7)$$

□ 基于贝叶斯定理， $P(c | \mathbf{x})$ 可写成

$$P(c | \mathbf{x}) = \frac{P(c)P(\mathbf{x} | c)}{P(\mathbf{x})} \quad (7.8)$$

类标记 c 相对于样本 $\mathbf{x}$ 的
“类条件概率” (class-
conditional probability),
或称“似然”。

先验概率
样本空间中各类样本所占的
比例，可通过各类样本出现
的频率估计（大数定理）

“证据” (evidence)
因子，与类标记无关

第7章 贝叶斯分类器

7.1 贝叶斯决策论



7.2 极大似然估计

7.3 朴素贝叶斯分类器

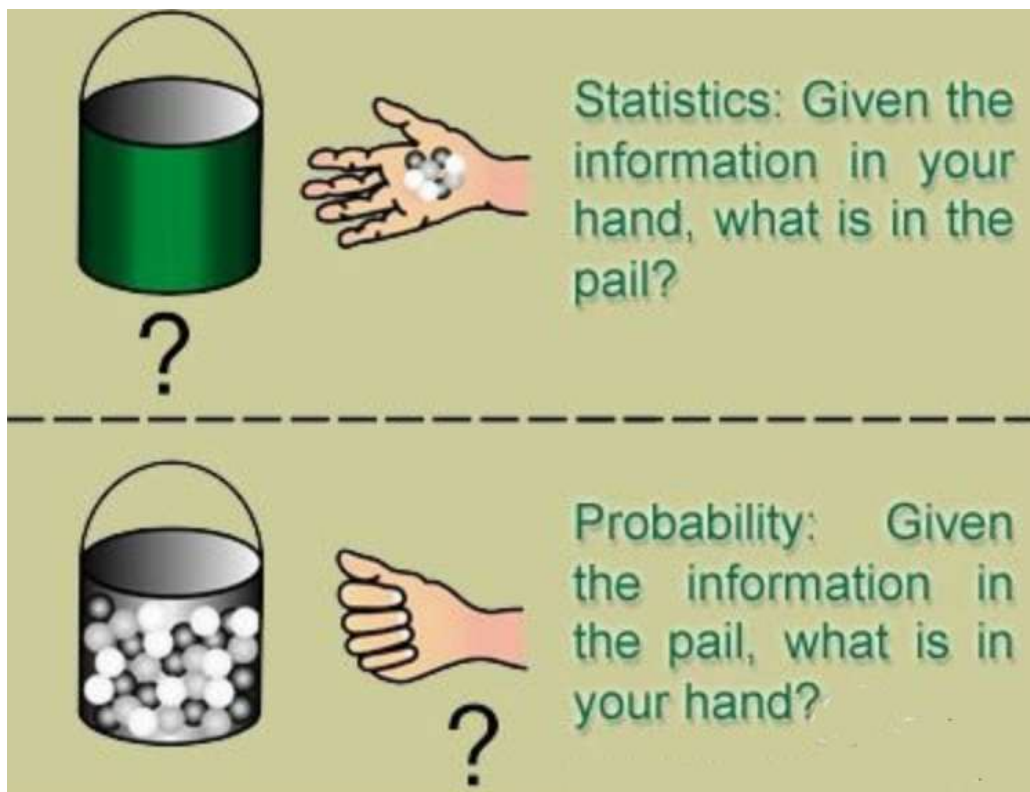
7.4 半朴素贝叶斯分类器

7.5 贝叶斯网

7.6 EM算法

1、概率 VS 统计

- 统计和概率是方法论上的区别，一个是归纳，一个是推理。
- 统计学：根据手中信息，猜猜桶里有啥？（样本归纳总结出总体）
- 概率论：根据桶中信息，猜猜手里有啥？（总体对样本进行预测）



1、概率 VS 统计

- 概率论: 有一个白箱子, 已知这个箱子的构造 (里面有几个红球、几个白球, 也就是所谓的分布函数), 然后计算下一个摸出来的球是红球的概率
- 统计学: 有一个黑箱子, 只看得到每次摸出来的是红球还是白球, 然后需要猜测这个黑箱子的内部结构, 例如红球和白球的比例是多少? (参数估计)
能不能认为红球40%, 白球60%? (假设检验)
- 概率论中的许多定理与结论, 如**大数定理**、**中心极限定理**等保证了统计推断的合理性。做统计推断一般都需要对那个黑箱子做各种各样的假设, 这些假设都是概率模型, **统计推断实际上就是在估计这些模型的参数**

1、概率 VS 统计

- 假设有一个袋子，里面装着白球和黑球，但是不知道他们分别有多少个，这时候需要估计每次取出一个球是白球的概率是多少？如何估计呢？可以通过连续有放回的从袋子里面取一百次，看看是白球还是黑球。假设取100次里面白球占70次，黑球30次。设抽取是白球的概率为 p 。那么一百次抽取的总概率为 $p(x;p)$

$$\begin{aligned} p(x;p) &= p(x_1, x_2, \dots, x_{100}; \theta) = p(x_1; \theta) * p(x_2; \theta) * \dots * p(x_{100}; \theta) \\ &= p^{70} * (1-p)^{30} \end{aligned}$$

$$\text{求导: } \log p(x;p) = \log p^{70} * (1-p)^{30}$$

- 令导数为零可以求出 $p=0.7$

2、极大似然估计

- **估计类条件概率的常用策略**：先假定其具有某种确定的概率分布形式，再基于训练样本对概率分布参数估计。在给定样本观测数据的情况下，选择使得这些数据出现概率（或概率密度）最大的参数值作为该参数的估计值。
- 记关于类别 c 的类条件概率为 $P(\mathbf{x} | c)$
 - ✓ 假设 $P(\mathbf{x} | c)$ 具有确定的形式被参数 θ_c 唯一确定，我们的任务就是利用训练集 D 估计参数 θ_c
- 概率模型的训练过程就是**参数估计过程**，统计学界的两个学派提供了不同的方案：
 - ✓ 频率主义学派 (frequentist) 认为参数虽然未知，但却存在客观值，因此可通过优化似然函数等准则来确定参数值
 - ✓ 贝叶斯学派 (Bayesian) 认为参数是未观察到的随机变量、其本身也可有分布，因此可假定参数服从一个先验分布，然后基于观测到的数据计算参数的后验分布。

2、极大似然估计—频率主义学派

□ 令 D_c 表示训练集中第 c 类样本的集合的集合，假设这些样本是独立的，则参数 θ_c 对于数据集 D_c 的似然是

$$P(D_c | \theta_c) = \prod_{\mathbf{x} \in D_c} P(\mathbf{x} | \theta_c) \quad (7.9)$$

- 对 θ_c 进行极大似然估计，寻找能最大化似然 $P(D_c | \theta_c)$ 的参数值 $\hat{\theta}_c$ 。直观上看，极大似然估计是试图在 θ_c 所有可能的取值中，找到一个使数据出现的“可能性”最大值。

□ 式 (7.9) 的连乘操作易造成下溢，通常使用对数似然(log-likelihood)

$$\begin{aligned} LL(\theta_c) &= \log P(D_c | \theta_c) \\ &= \sum_{\mathbf{x} \in D_c} \log P(\mathbf{x} | \theta_c) \end{aligned} \quad (7.10)$$

□ 此时参数 θ_c 的极大似然估计 $\hat{\theta}_c$ 为 $\hat{\theta}_c = \operatorname{argmax}_{\theta_c} LL(\theta_c)$ (7.11)

2、极大似然估计

- 例如：在连续属性情形下，假设概率密度函数 $p(\mathbf{x} | c) \sim N(\boldsymbol{\mu}_c, \boldsymbol{\sigma}_c^2)$ ，则参数 $\boldsymbol{\mu}_c$ 和 $\boldsymbol{\sigma}_c^2$ 的极大似然估计为

$$\hat{\boldsymbol{\mu}}_c = \frac{1}{|D_c|} \sum_{\mathbf{x} \in D_c} \mathbf{x} \quad (7.12)$$

$$\hat{\boldsymbol{\sigma}}_c^2 = \frac{1}{|D_c|} \sum_{\mathbf{x} \in D_c} (\mathbf{x} - \hat{\boldsymbol{\mu}}_c)(\mathbf{x} - \hat{\boldsymbol{\mu}}_c)^T \quad (7.13)$$

- 也就是说，通过极大似然法得到的正态分布均值就是样本均值，方差就是 $(\mathbf{x} - \hat{\boldsymbol{\mu}}_c)(\mathbf{x} - \hat{\boldsymbol{\mu}}_c)^T$ 的均值，这显然是一个符合直觉的结果。
- 需注意的是，这种参数化的方法虽能使类条件概率估计变得相对简单，但估计结果的准确性严重依赖于所假设的概率分布形式是否符合潜在的真实数据分布。

第7章 贝叶斯分类器

7.1 贝叶斯决策论

7.2 极大似然估计

➡ 7.3 朴素贝叶斯分类器

7.4 半朴素贝叶斯分类器

7.5 贝叶斯网

7.6 EM算法

1、朴素贝叶斯分类器

- 估计后验概率 $P(c | \mathbf{x})$ 主要困难：类条件概率 $P(\mathbf{x} | c)$ 是所有属性上的联合概率，难以从有限的训练样本估计获得。
- 朴素贝叶斯分类器(Naive Bayes Classifier)采用了“属性条件独立性假设”(attribute conditional independence assumption)：每个属性独立地对分类结果发生影响。
- 基于属性条件独立性假设，(7.8)可重写为

$$P(c | \mathbf{x}) = \frac{P(c)P(\mathbf{x} | c)}{P(\mathbf{x})} = \frac{P(c)}{P(\mathbf{x})} \prod_{i=1}^d P(x_i | c) \quad (7.14)$$

- 其中 d 为属性数目， x_i 为 $\mathbf{x}$ 在第 i 个属性上的取值。

x_i 实际上是一个“属性-值”对，例如“色泽=青绿”。为便于讨论，在上下文明确时，有时我们用 x_i 表示第 i 个属性对应的变量(如“色泽”)，有时直接用其指代 $\mathbf{x}$ 在第 i 个属性上的取值(如“青绿”)。

1、朴素贝叶斯分类器

$$P(c | \mathbf{x}) = \frac{P(c)P(\mathbf{x} | c)}{P(\mathbf{x})} = \frac{P(c)}{P(\mathbf{x})} \prod_{i=1}^d P(x_i | c) \quad (7.14)$$

由于对所有类别来说 $P(\mathbf{x})$ 相同，因此基于式 (7.6) 的贝叶斯判定准则有

$$h_{nb}(\mathbf{x}) = \operatorname{argmax}_{c \in y} P(c) \prod_{i=1}^d P(x_i | c) \quad (7.15)$$

- 这就是朴素贝叶斯分类器的表达式。

1、朴素贝叶斯分类器

□ 朴素贝叶斯分类器的训练器的训练过程就是基于训练集 D 来估计类先验概率 $P(c)$ ，并为每个属性估计条件概率 $P(x_i | c)$ 。

- 令 D_c 表示训练集 D 中第 c 类样本组合的集合，若有充足的独立同分布样本，则可容易地估计出类先验概率

$$P(c) = \frac{|D_c|}{D} \quad (7.16)$$

- 对离散属性而言，令 D_{c,x_i} 表示 D_c 中在第 i 个属性上取值为 x_i 的样本组成的集合，则条件概率 $P(x_i | c)$ 可估计为

$$P(x_i | c) = \frac{|D_{c,x_i}|}{D} \quad (7.17)$$

- 对连续属性 i 而言可考虑概率密度函数，假定 $p(x_i | c) \sim N(\mu_{c,i}, \sigma_{c,i}^2)$ ，其中 $\mu_{c,i}$ 和 $\sigma_{c,i}^2$ 分别是第 c 类样本在第 i 个属性上取值的均值和方差，则有

$$P(x_i | c) = \frac{1}{\sqrt{2\pi}\sigma_{c,i}} \exp\left(-\frac{(x_i - \mu_{c,i})^2}{2\sigma_{c,i}^2}\right) \quad (7.18)$$

1、朴素贝叶斯分类器

- 例子：用西瓜数据集3.0训练一个朴素贝叶斯分类器，对测试例“测1”进行分类 (p151, 西瓜数据集 p84 表4.3)

表 4.3 西瓜数据集 3.0

编号	色泽	根蒂	敲声	纹理	脐部	触感	密度	含糖率	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	0.697	0.460	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	0.774	0.376	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	0.634	0.264	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	0.608	0.318	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	0.556	0.215	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	0.403	0.237	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	0.481	0.149	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	0.437	0.211	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	0.666	0.091	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	0.243	0.267	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	0.245	0.057	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	0.343	0.099	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	0.639	0.161	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	0.657	0.198	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	0.360	0.370	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	0.593	0.042	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	0.719	0.103	否

编号	色泽	根蒂	敲声	纹理	脐部	触感	密度	含糖率	好瓜
测 1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	0.697	0.460	?

1、朴素贝叶斯分类器

□ 首先估计类先验概率 $P(c)$ ，显然有：

$$P(\text{好瓜} = \text{是}) = \frac{8}{17} \approx 0.471, \quad P(\text{好瓜} = \text{否}) = \frac{9}{17} \approx 0.529.$$

□ 然后，为每个属性估计条件概率 $P(x_i | c)$ ：

$$P_{\text{青绿}|\text{是}} = P(\text{色泽} = \text{青绿} | \text{好瓜} = \text{是}) = \frac{3}{8} = 0.375,$$

$$P_{\text{青绿}|\text{否}} = P(\text{色泽} = \text{青绿} | \text{好瓜} = \text{否}) = \frac{3}{9} \approx 0.333,$$

$$P_{\text{蜷缩}|\text{是}} = P(\text{根蒂} = \text{蜷缩} | \text{好瓜} = \text{是}) = \frac{5}{8} = 0.375,$$

$$P_{\text{蜷缩}|\text{否}} = P(\text{根蒂} = \text{蜷缩} | \text{好瓜} = \text{否}) = \frac{3}{9} \approx 0.333,$$

$$P_{\text{浊响}|\text{是}} = P(\text{敲声} = \text{浊响} | \text{好瓜} = \text{是}) = \frac{6}{8} = 0.750,$$

$$P_{\text{浊响}|\text{否}} = P(\text{敲声} = \text{浊响} | \text{好瓜} = \text{否}) = \frac{4}{9} \approx 0.444,$$

1、朴素贝叶斯分类器

$$\begin{aligned}P_{\text{清晰}|\text{是}} &= P(\text{纹理} = \text{清晰} | \text{好瓜} = \text{是}) = \frac{7}{8} = 0.875, \\P_{\text{清晰}|\text{否}} &= P(\text{纹理} = \text{清晰} | \text{好瓜} = \text{否}) = \frac{2}{9} \approx 0.222, \\P_{\text{凹陷}|\text{是}} &= P(\text{脐部} = \text{凹陷} | \text{好瓜} = \text{是}) = \frac{6}{8} = 0.750, \\P_{\text{凹陷}|\text{否}} &= P(\text{脐部} = \text{凹陷} | \text{好瓜} = \text{否}) = \frac{2}{9} \approx 0.222, \\P_{\text{硬滑}|\text{是}} &= P(\text{触感} = \text{硬滑} | \text{好瓜} = \text{是}) = \frac{6}{8} = 0.750, \\P_{\text{硬滑}|\text{否}} &= P(\text{触感} = \text{硬滑} | \text{好瓜} = \text{否}) = \frac{6}{9} \approx 0.667,\end{aligned}$$

1、朴素贝叶斯分类器

$$\begin{aligned} p_{\text{密度}:0.697|\text{是}} &= p(\text{密度} = 0.697 \mid \text{好瓜} = \text{是}) \\ &= \frac{1}{\sqrt{2\pi} \cdot 0.129} \exp\left(-\frac{(0.697 - 0.574)^2}{2 \cdot 0.129^2}\right) \approx 1.959, \end{aligned}$$

$$\begin{aligned} p_{\text{密度}:0.697|\text{否}} &= p(\text{密度} = 0.697 \mid \text{好瓜} = \text{否}) \\ &= \frac{1}{\sqrt{2\pi} \cdot 0.195} \exp\left(-\frac{(0.697 - 0.496)^2}{2 \cdot 0.195^2}\right) \approx 1.203, \end{aligned}$$

$$\begin{aligned} p_{\text{含糖}:0.460|\text{是}} &= p(\text{含糖率} = 0.460 \mid \text{好瓜} = \text{是}) \\ &= \frac{1}{\sqrt{2\pi} \cdot 0.101} \exp\left(-\frac{(0.460 - 0.279)^2}{2 \cdot 0.101^2}\right) \approx 0.788, \end{aligned}$$

$$\begin{aligned} p_{\text{含糖}:0.460|\text{否}} &= p(\text{含糖率} = 0.460 \mid \text{好瓜} = \text{否}) \\ &= \frac{1}{\sqrt{2\pi} \cdot 0.108} \exp\left(-\frac{(0.460 - 0.154)^2}{2 \cdot 0.108^2}\right) \approx 0.066. \end{aligned}$$

1、朴素贝叶斯分类器

```
>>> import numpy as np
>>> a = np.array([0.697, 0.774, 0.634, 0.608, 0.556, 0.403, 0.481, 0.437])
>>> a.mean()
0.57374999999999998
>>> a.std(ddof=1)
0.12921051483086482
>>> b = np.array([0.666, 0.243, 0.245, 0.343, 0.639, 0.657, 0.360, 0.593, 0.719])
>>> b.mean()
0.49611111111111117
>>> b.std(ddof=1)
0.19471867170641627
>>> c = np.array([0.460, 0.376, 0.264, 0.318, 0.215, 0.237, 0.149, 0.211])
>>> c.mean()
0.27875
>>> c.std(ddof=1)
0.10092394590553255
>>> d = np.array([0.091, 0.267, 0.057, 0.099, 0.161, 0.198, 0.370, 0.042, 0.103])
>>> d.mean()
0.15422222222222221
>>> d.std(ddof=1)
0.10779468653159321
```

一、朴素贝叶斯分类器

□ 于是，有：

$$P(\text{好瓜} = \text{是}) \times P(\text{青绿}|\text{是}) \times P(\text{蜷缩}|\text{是}) \times P(\text{浊响}|\text{是}) \times P(\text{清晰}|\text{是}) \\ \times P(\text{凹陷}|\text{是}) \times P(\text{硬滑}|\text{是}) \times p_{\text{密度}:0.697|\text{是}} \times p_{\text{含糖}:0.460|\text{是}} \approx 0.038$$

□ 由于 $0.038 > 6.80 \times 10^{-5}$ ，因此，朴素贝叶斯分类器将测试样本“测1”判别为“好瓜”

2、拉普拉斯修正

- 需要注意：若某个属性值在训练集中没有与某个类同时出现过，则直接计算会出现问题，比如“敲声=清脆”测试例，训练集中没有该样例，因此连乘式计算的概率值为0，无论其他属性上明显像好瓜，分类结果都是“好瓜=否”，这显然不合理。

$$P_{\text{清脆}|\text{是}} = P(\text{敲声} = \text{清脆} | \text{好瓜} = \text{是}) = \frac{0}{8} = 0$$

- 为了避免其他属性携带的信息被训练集中未出现的属性值“抹去”，在估计概率值时通常要进行“平滑”，常用“拉普拉斯修正” (Laplacian correction)
- 令 N 表示训练集 D 中可能的类别数， N_i 表示第 i 个属性可能的取值数，则式 (7.16) 和 (7.17) 分别修正为

$$\hat{P}(c) = \frac{|D_c| + 1}{|D| + N} \quad (7.19)$$

$$\hat{P}(x_i | c) = \frac{|D_{c,x_i}| + 1}{|D| + N_i} \quad (7.20)$$

2、拉普拉斯修正

□ 例如，在本节的例子中，类先验概率可估计为

$$\hat{P}(\text{好瓜} = \text{是}) = \frac{8+1}{17+2} \approx 0.474, \hat{P}(\text{好瓜} = \text{否}) = \frac{9+1}{17+2} \approx 0.526.$$

□ 类似地， $P_{\text{青绿}|\text{是}}$ 和 $P_{\text{青绿}|\text{否}}$ 可估计为

$$\hat{P}_{\text{青绿}|\text{是}} = \hat{P}(\text{色泽} = \text{青绿} | \text{好瓜} = \text{是}) = \frac{3+1}{8+3} \approx 0.364$$

$$\hat{P}_{\text{青绿}|\text{否}} = \hat{P}(\text{色泽} = \text{青绿} | \text{好瓜} = \text{否}) = \frac{3+1}{9+3} \approx 0.333$$

□ 同时，上文提到的概率 $P_{\text{清脆}|\text{是}}$ 可估计为

$$\hat{P}_{\text{清脆}|\text{是}} = \hat{P}(\text{敲声} = \text{清脆} | \text{好瓜} = \text{是}) = \frac{0+1}{8+3} \approx 0.091$$

2、拉普拉斯修正

- 显然：拉普拉斯修正避免了因训练样本不充分而导致概率估值为0的问题
- 当训练集变大时，修正过程所引入的先验的影响也会逐渐变得可忽略，使得估值趋向于实际概率值。

- 在现实任务中朴素贝叶斯分类器有多种使用方式。
 - ✓ 例如，若任务对预测速度要求较高，则对给定训练集，可将朴素贝叶斯分类器涉及的所有概率估值事先计算好存储起来，这样在进行预测时只需“查表”即可进行判别；
 - ✓ 若任务据更替频繁，则可采用“懒惰学习” (lazy learning) 方式，先不进行任何训练，待收到预测请求时再根据当前数据集进行概率估值；
 - ✓ 若数据不断增加，则可在 现有估值基础上，仅对新增样本的属性值所涉及的概率估值进行计数修正即可实现增量学习。

第7章 贝叶斯分类器

7.1 贝叶斯决策论

7.2 极大似然估计

7.3 朴素贝叶斯分类器

➡ 7.4 半朴素贝叶斯分类器

7.5 贝叶斯网

7.6 EM算法

1、半朴素贝叶斯分类器

- ❑ 为了降低贝叶斯公式中估计后验概率的困难，朴素贝叶斯分类器采用的属性条件独立性假设，但现实中，这个条件往往难成立。对属性条件独立假设进行一定程度的放松，由此产生了一类称为“半朴素贝叶斯分类器”（semi-naïve Bayes classifiers）
- ❑ **半朴素贝叶斯分类器**：适当考虑一部分属性间的相互依赖信息，从而既不需进行完全联合概率计算，又不至于彻底忽略了比较强的属性依赖关系
- ❑ 最常用的一种策略：“独依赖估计”（One-Dependent Estimator, 简称ODE），**假设每个属性在类别之外最多仅依赖一个其他属性**，即

$$P(c | x) \propto P(c) \prod_{i=1}^d P(x_i | c, pa_i)$$

- 其中 pa_i 为属性 x_i 所依赖的属性，称为 x_i 的父属性
- ❑ 对每个属性 x_i ，若其父属性 pa_i 已知，则可估计概率值 $P(x_i | c, pa_i)$ ，于是问题的关键转化为如何确定每个属性的父属性，不同做法产生不同的独依赖分类器。

2、SPODE(Super-Parent ODE)方法

- 最直接的做法是假设所有属性都依赖于同一属性，称为“超父” (super-parent), 然后通过交叉验证等模型选择方法来确定超父属性，由此形成了 SPODE (Super-Parent ODE)方法。

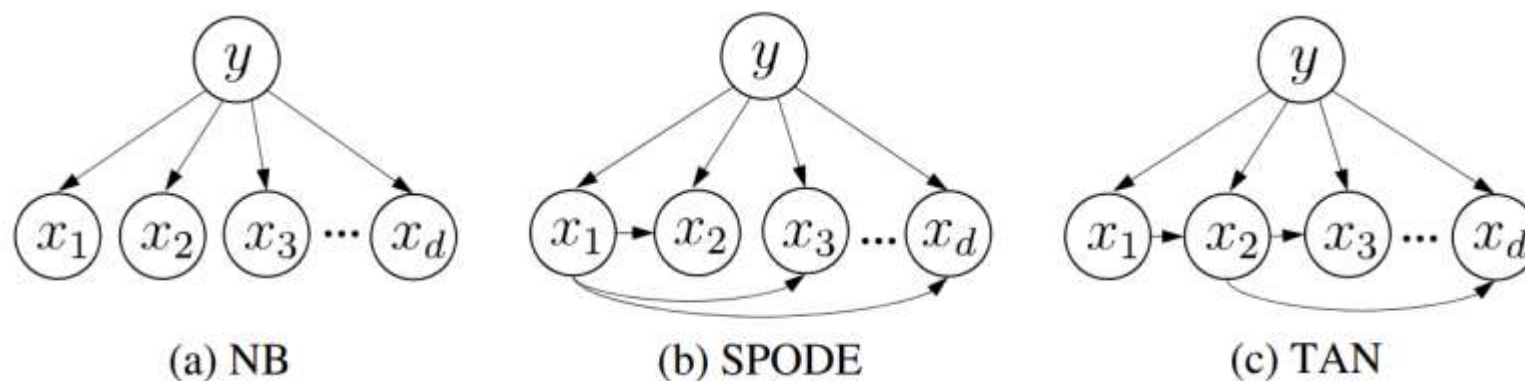


图7.1 朴素贝叶斯分类器与两种半朴素分类器所考虑的属性依赖关系

- 在图7.1 (b)中, x_1 是超父属性。

3、TAN(Tree Augmented naive Bayes)

□ TAN (Tree augmented Naive Bayes) [Friedman et al., 1997] 则在最大带权生成树 (Maximum weighted spanning tree) 算法 [Chow and Liu, 1968] 的基础上, 通过以下步骤将属性间依赖关系简约为图7.1 (c)。

① 计算任意两个属性之间的条件互信息 (conditional mutual information)

$$I(x_i, x_j | y) = \sum_{x_i, x_j; c \in y} P(x_i, x_j | c) \log \frac{P(x_i, x_j | c)}{P(x_i | c)P(x_j | c)}$$

② 以属性为结点构建完全图, 任意两个结点之间边的权重设为 $I(x_i, x_j | y)$

③ 构建此完全图的最大带权生成树, 挑选根变量, 将边设为有向;

④ 加入类别节点y, 增加从y到每个属性的有向边。

✓ 条件互信息 $I(x_i, x_j | y)$ 刻画了属性 x_i 和 x_j 之间在已知类别情况下的相关性。

✓ 因此, 通过最大生成树算法, TAN实际保留了强相关属性之间的依赖性。

□ AODE (Averaged One-Dependent Estimator) [Webb et al. 2005] 是一种基于集成学习机制、更为强大的分类器。

- 尝试将每个属性作为超父来构建 SPODE
- 将具有足够训练数据支撑的SPODE集成起来作为最终结果

$$P(c | \mathbf{x}) \propto \sum_{i=1; |D_{x_i}| \geq m'}^d P(c, x_i) \prod_{j=1}^d P(x_j | c, x_i)$$

其中, D_{x_i} 是在第 i 个属性上取值 x_i 的样本的集合, m' 为阈值常数。显然, AODE需估计 $P(c, x_i)$ 和 $P(x_j | c, x_i)$

$$\hat{P}(x_i, c) = \frac{|D_{c, x_i}| + 1}{|D| + N_i} \quad \hat{P}(x_j | c, x_i) = \frac{|D_{c, x_i, x_j}| + 1}{|D_{c, x_i}| + N_j}$$

其中, N_i 是在第 i 个属性上可能取值数, D_{c, x_i} 是类别为 c 且在第 i 个属性上取值为 x_i 的样本集合, D_{c, x_i, x_j} 是类别为 c 且在第 i 和第 j 个属性上取值分别为 x_i 和 x_j 的样本集合。

第7章 贝叶斯分类器

7.1 贝叶斯决策论

7.2 极大似然估计

7.3 朴素贝叶斯分类器

7.4 半朴素贝叶斯分类器

➡ 7.5 贝叶斯网

7.6 EM算法

1、贝叶斯网

- 贝叶斯网 (Bayesian network) 亦称“信念网”(belief network), 它借助**有向无环图** (Directed Acyclic Graph, DAG) 来**刻画属性间的依赖关系**, 并使用条件概率表 (Conditional Probability Table, CPT) 来表述属性的联合概率分布。
- 一个贝叶斯网 B 由结构 G 和参数 Θ 两部分构成, 即: $B = \langle G, \Theta \rangle$
- 网络结构 G 是一个有向无环图, 其每个节点对应于一个属性若两个属性有直接依赖关系, 则它们由一条边连接起来。
- 参数 Θ 定量描述这种依赖关系。
- 假设属性 x_i 在 G 中的父结点集为 π_i , 则 Θ 含了每个属性的条件概率表

$$\theta_{x_i|\pi_i} = P_B(x_i | \pi_i)$$

1、贝叶斯网

图7.2例子：

- ✓ 从网络图结构可以看出：“色泽”直接依赖于“好瓜”和“甜度”，而“根蒂”则直接依赖于“甜度”
- ✓ 从条件概率表可以得到：“根蒂”对“甜度”的量化依赖关系， $P(\text{根蒂} = \text{硬挺} | \text{甜度} = \text{高}) = 0.1$

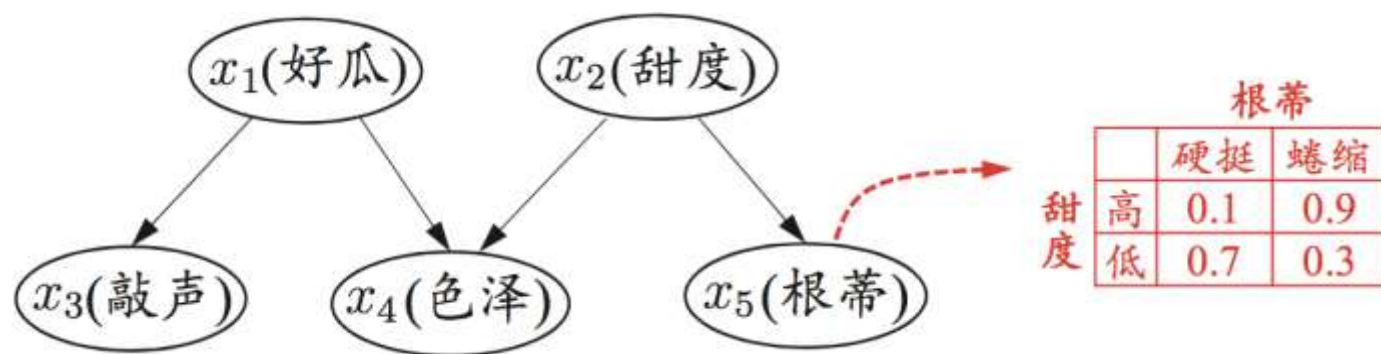


图7.2 西瓜问题的一种贝叶斯网结构以及属性“根蒂”的条件概率表

2、贝叶斯网：结构

□ 贝叶斯网有效地表达了属性间的条件独立性。给定父结集，贝叶斯网假设每个属性与他的非后裔属性独立。

□ 于是， $B = \langle G, \Theta \rangle$ 将属性 $x_1, x_2, \dots, x_d$ 的联合概率分布定义为

$$P_B(x_1, x_2, \dots, x_d) = \prod_{i=1}^d P_B(x_i \mid \pi_i) = \prod_{i=1}^d \theta_{x_i \mid \pi_i} \quad (7.26)$$

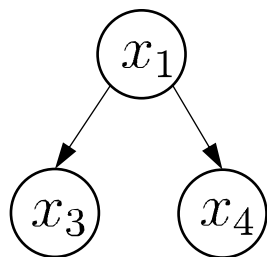
图7.2的联合概率分布定义为：

$$P(x_1, x_2, x_3, x_4, x_5) = P(x_1)P(x_2)P(x_3 \mid x_1)P(x_4 \mid x_1, x_2)P(x_5 \mid x_2)$$

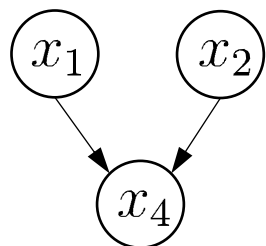
显然， x_3 和 x_4 在给定 x_1 的取值时独立， x_4 和 x_5 在给定 x_2 的取值时独立，分别简记为：记为 $x_3 \perp x_4 \mid x_1$ 和 $x_4 \perp x_5 \mid x_2$ 。

2、贝叶斯网：结构

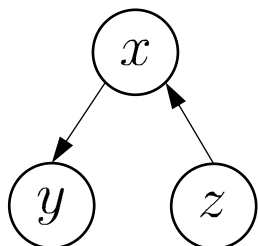
□ 下图显示出贝叶斯网中三个变量之间的典型依赖关系：



同父结构



V型结构



顺序结构

在“同父” (common parent) 结构中, 给定父结点 x_1 的取值, 则 x_3 与 x_4 条件独立. 在“顺序”结构中, 给定 x 的值, 则 y 与 z 条件独立. V 型结构(V-structure)亦称“冲撞”结构, 给定子结点 x_4 的取值, x_1 与 x_2 必不独立; 奇妙的是, 若 x_4 的取值完全未知, 则 V 型结构下 x_1 与 x_2 却是相互独立的. 我们做一个简单的验证:

$$\begin{aligned} P(x_1, x_2) &= \sum_{x_4} P(x_1, x_2, x_4) \\ &= \sum_{x_4} P(x_4 \mid x_1, x_2) P(x_1) P(x_2) \\ &= P(x_1) P(x_2) . \end{aligned} \tag{7.27}$$

这样的独立性称为“边际独立性” (marginal independence), 记为 $x_1 \perp\!\!\!\perp x_2$.

2、贝叶斯网：结构

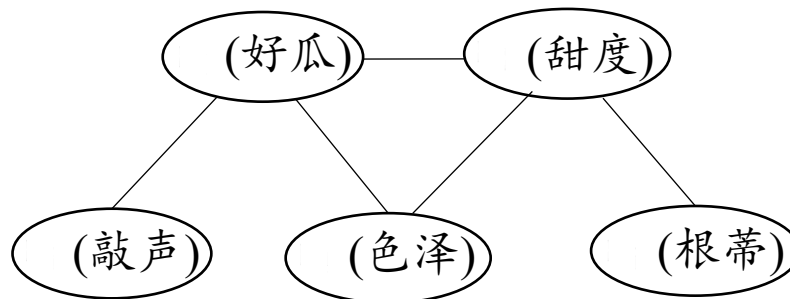
□ 分析有向图中变量间的条件独立性，可使用“有向分离”

(D-separation)

- V型结构父结点相连
- 有向边变成无向边

由此产生的图称为道德图

(moral graph)



3、贝叶斯网：学习

- 贝叶斯网络首要任务：根据训练集找出结构最“恰当”的贝叶斯网。
- 用评分函数评估贝叶斯网与训练数据的契合程度。
 - “最小描述长度” (Minimal Description Length, MDL) 综合编码长度（包括描述网路和编码数据）最短（类似前缀编码）
- 给定训练集 $D = \{x_1, x_2, \dots, x_m\}$ ，贝叶斯网 $B = \langle G, \Theta \rangle$ 在 D 上的评价函数可以写为

$$S(B \mid D) = f(\theta)|B| - LL(B \mid D) \quad (7.28)$$

- 其中, $|B|$ 是贝叶斯网的参数个数; $f(\theta)$ 表示描述每个参数 θ 所需的字节数, 而

$$LL(B \mid D) = \sum_{i=1}^m \log P_B(x_i) \quad (7.29)$$

- 是贝叶斯网的对数似然。

3、贝叶斯网：推断

- 通过已知变量观测值来推测待推测查询变量的过程称为“推断”(inference)，已知变量观测值称为“证据”(evidence)。
- 最理想的是根据贝叶斯网络定义的联合概率分布来精确计算后验概率，在现实应用中，贝叶斯网的近似推断常使用吉布斯采样(Gibbs sampling)来完成。
- 吉布斯采样随机产生一个与证据 $E = e$ 一致的样本 q^0 作为初始点，然后每步从当前样本出发产生下一个样本。假定经过 T 次采样的得到与 q 一致的样本共有 n_q 个，则可近似估算出后验概率

$$P(Q = q \mid E = e) \simeq \frac{n_q}{T} \quad (7.33)$$

- 吉布斯采样可以看做，每一步仅依赖于前一步的状态，这是一个“马尔可夫链”(Markov Chain)。更多马尔可夫链和吉布斯采样内容参见14.5章节。

第7章 贝叶斯分类器

7.1 贝叶斯决策论

7.2 极大似然估计

7.3 朴素贝叶斯分类器

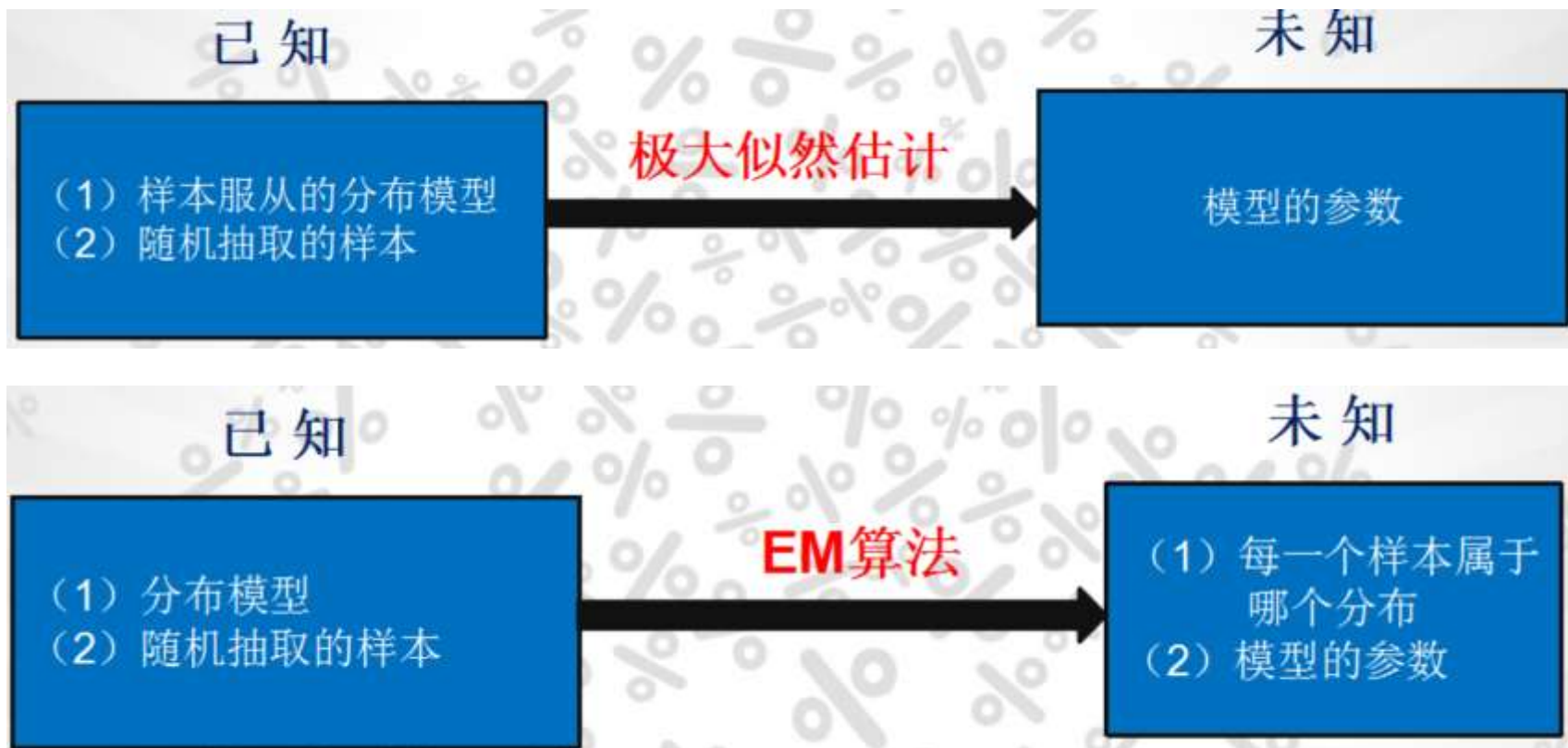
7.4 半朴素贝叶斯分类器

7.5 贝叶斯网

➡ 7.6 EM算法

一、EM算法

- 如果使用基于最大似然估计的模型，模型中存在**隐变量**，就要用EM算法做参数估计。



EM算法

- “不完整”的样本：西瓜已经脱落的根蒂，无法看出是“蜷缩”还是“坚挺”，则训练样本的“根蒂”属性变量值未知，如何计算？

EM算法

- “不完整”的样本：西瓜已经脱落的根蒂，无法看出是“蜷缩”还是“坚挺”，则训练样本的“根蒂”属性变量值未知，如何计算？
- 未观测的变量称为“**隐变量**” (latent variable)。令 $\mathbf{X}$ 表示已观测变量集, $\mathbf{Z}$ 表示隐变量集，如果想对模型参数 Θ 做极大似然估计，则应最大化对数似然函数

$$LL(\Theta \mid \mathbf{X}, \mathbf{Z}) = \ln P(\mathbf{X}, \mathbf{Z} \mid \Theta) \quad (7.34)$$

EM算法

- “不完整”的样本：西瓜已经脱落的根蒂，无法看出是“蜷缩”还是“坚挺”，则训练样本的“根蒂”属性变量值未知，如何计算？
- 未观测的变量称为“隐变量”（latent variable）。令 $\mathbf{X}$ 表示已观测变量集， $\mathbf{Z}$ 表示隐变量集，若预对模型参数 Θ 做极大似然估计，则应最大化对数似然函数

$$LL(\Theta | \mathbf{X}, \mathbf{Z}) = \ln P(\mathbf{X}, \mathbf{Z} | \Theta) \quad (7.34)$$

- 由于 $\mathbf{Z}$ 是隐变量，上式无法直接求解。此时我们可以通过对 $\mathbf{Z}$ 计算期望，来最大化已观测数据的对数“边际似然”（marginal likelihood）

$$LL(\Theta | \mathbf{X}) = \ln P(\mathbf{X} | \Theta) = \ln \sum_{\mathbf{Z}} P(\mathbf{X}, \mathbf{Z} | \Theta) \quad (7.35)$$

EM算法

EM (**Expectation-Maximization**)算法 [Dempster et al., 1977] 是常用的**估计参数隐变量**的利器。

- 当参数 Θ 已知 $\rightarrow$ 根据训练数据推断出最优隐变量 $\mathbf{Z}$ 的值 (E步)
- 当 $\mathbf{Z}$ 已知 $\rightarrow$ 对 Θ 做极大似然估计 (M步)

EM算法

EM (Expectation-Maximization)算法 [Dempster et al., 1977] 是常用的估计参数隐变量的利器。

- 当参数 Θ 已知 $\rightarrow$ 根据训练数据推断出最优隐变量 $\mathbf{Z}$ 的值 (E步)
- 当 $\mathbf{Z}$ 已知 $\rightarrow$ 对 Θ 做极大似然估计 (M步)

于是，以初始值 Θ^0 为起点，对式子 (7.35)，可迭代执行以下步骤直至收敛：

- ✓ 基于 Θ^t 推断隐变量 $\mathbf{Z}$ 的期望，记为 $\mathbf{Z}^t$ ；
- ✓ 基于已观测到变量 $\mathbf{x}$ 和 $\mathbf{Z}^t$ 对参数 Θ 做极大似然估计，记为 Θ^{t+1} ；

- 这就是EM算法的原型。

EM算法

进一步，若我们不是取 Z 的期望，而是基于 Θ^t 计算隐变量 Z 的概率分布 $P(Z | X, \Theta^t)$ 则EM算法的两个步骤是：

□ **E步 (Expectation)** : 以当前参数 Θ^t 推断隐变量分布 $P(Z | X, \Theta^t)$ ，并计算对数似然 $LL(\Theta | X, Z)$ 关于 Z 的期望：

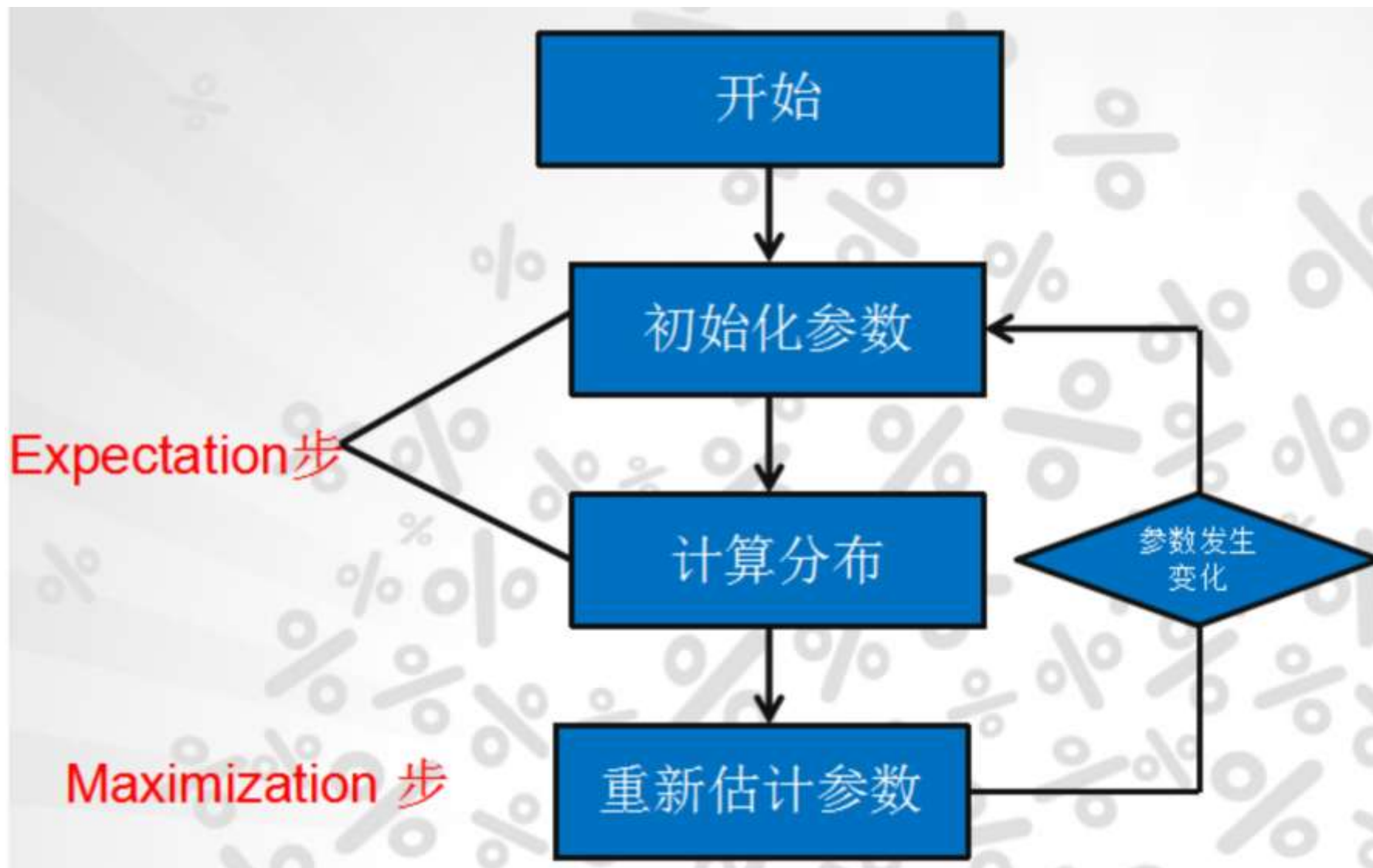
$$Q(\Theta | \Theta^t) = \mathbb{E}_{Z|X, \Theta^t} LL(\Theta | X, Z) \quad (7.36)$$

□ **M步 (Maximization)** : 寻找参数最大化期望似然，即

$$\Theta^{t+1} = \underset{\Theta}{\operatorname{argmax}} Q(\Theta | \Theta^t) \quad (7.37)$$

□ **EM算法使用两个步骤交替计算**：第一步计算期望(**E步**)，利用当前估计的参数值计算对数似然的参数值；第二步最大化(**M步**)，寻找能使**E步**产生的似然期望最大化的参数值……直至收敛到全局最优解。

一、EM算法



二、EM算法举例

□ 假设现在有两枚硬币1和2,随机抛掷后正面朝上概率分别为 P_1 , P_2 。为了估计这两个概率, 做实验, 每次取一枚硬币, 连掷5下, 记录下结果, 如下表, 可以很容易地估计出 **P_1 和 P_2**

硬币	结果	统计
1	正正反正反	3正-2反
2	反反正正反	2正-3反
1	正反反反反	1正-4反
2	正反反正正	3正-2反
1	反正正反反	2正-3反

$$P_1 = (3+1+2) / 15 = 0.4$$

$$P_2 = (2+3) / 10 = 0.5$$

二、EM算法举例

□ 还是上面的问题，抹去每轮投掷时使用的硬币标记，如下

硬币	结果	统计
Unknown	正正反正反	3正-2反
Unknown	反反正正反	2正-3反
Unknown	正反反反反	1正-4反
Unknown	正反反正正	3正-2反
Unknown	反正正反反	2正-3反

□ 目标没变，还是估计**P1**和**P2**，要怎么做呢？

二、EM算法举例

- 还是上面的问题，抹去每轮投掷时使用的硬币标记，如下
- 显然此时多了一个隐变量 z ，可以把它认为是一个5维的向量 $(z_1, z_2, z_3, z_4, z_5)$ ，代表每次投掷时所使用的硬币，比如 z_1 ，就代表第一轮投掷时使用的硬币是1还是2。但是，这个变量 z 不知道，就无法去估计 P_1 和 P_2 ，所以，**必须先估计出 z ，然后才能进一步估计 P_1 和 P_2 。**
- 但要**估计 z ，又得知道 P_1 和 P_2** ，这样才能用最大似然概率法则去估计 z ，这不是鸡生蛋和蛋生鸡的问题吗，如何破？
- **答案就是先随机初始化一个 P_1 和 P_2** ，用它来估计 z ，然后基于 z ，还是按照最大似然概率法则去估计新的 P_1 和 P_2 。
- 如果新的 P_1 和 P_2 和初始化的 P_1 和 P_2 一样，请问这说明了什么？这说明**初始化的 P_1 和 P_2 是一个相当靠谱的估计！**

二、EM算法举例

□ 就是说，初始化的 $P1$ 和 $P2$ ，按照最大似然概率就可以估计出 z ，然后基于 z ，按照最大似然概率可以反过来估计出 $P1$ 和 $P2$ ，当与初始化的 $P1$ 和 $P2$ 一样时，说明是 $P1$ 和 $P2$ 很有可能就是真实的值。这里面包含了两个交互的最大似然估计。

□ 如果新估计出来的 $P1$ 和 $P2$ 和初始化的值差别很大，怎么办呢？就是继续用新的 $P1$ 和 $P2$ 迭代，直至收敛。

二、EM算法举例

□ 不妨先随便给P1和P2赋一个值，比如：

$$P1 = 0.2$$

$$P2 = 0.7$$

□ 然后，看看第一轮抛掷最可能是哪个硬币。

如果是硬币1，得出3正2反的概率为 $0.2*0.2*0.2*0.8*0.8 = 0.00512$

如果是硬币2，得出3正2反的概率为 $0.7*0.7*0.7*0.3*0.3=0.03087$

然后依次求出其他4轮中的相应概率。做成表格如下：

二、EM算法举例

轮数	若是硬币1	若是硬币2
1	0.00512	0.03087
2	0.02048	0.01323
3	0.08192	0.00567
4	0.00512	0.03087
5	0.02048	0.01323

□ 按照最大似然法则：

第1轮中最有可能的是硬币2

第2轮中最有可能的是硬币1

第3轮中最有可能的是硬币1

第4轮中最有可能的是硬币2

第5轮中最有可能的是硬币1

二、EM算法举例

□ 把上面的值作为 z 的估计值。然后按照最大似然概率法则来估计新的 $P1$ 和 $P2$ 。

$$P1 = (2+1+2) / 15 = 0.33$$

$$P2 = (3+3) / 10 = 0.6$$

□ 设想本身已经知道每轮抛掷时的硬币就是如标签已知的表格标示的那样，那么， $P1$ 和 $P2$ 的最大似然估计就是0.4和0.5（将这两个值称为 $P1$ 和 $P2$ 的真实值）。那么对比下初始化的 $P1$ 和 $P2$ 和新估计出的 $P1$ 和 $P2$ ：

初始化的P1	估计出的P1	真实的P1	初始化的P2	估计出的P2	真实的P2
0.2	0.33	0.4	0.7	0.6	0.5

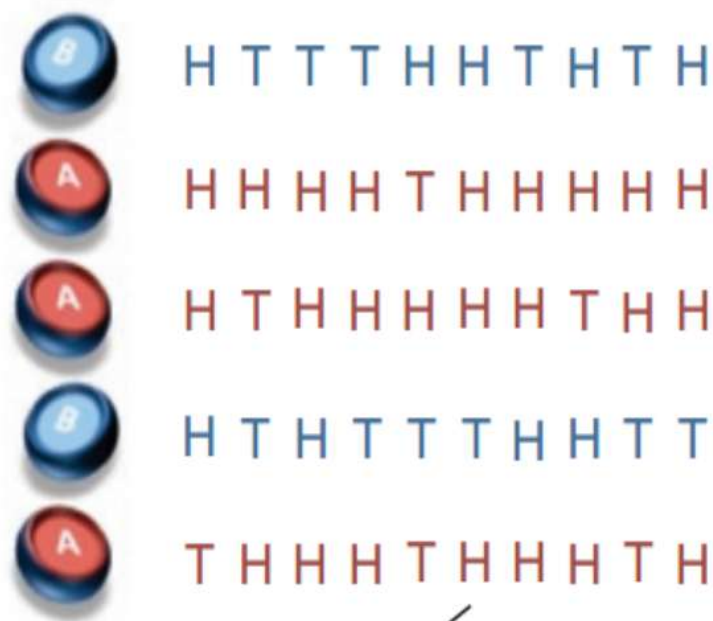
二、EM算法举例

- 估计的 $P1$ 和 $P2$ 相比于它们的初始值，更接近它们的真实值！
- 可以期待，继续按照上面的思路，用估计出的 $P1$ 和 $P2$ 再来估计 z ，再用 z 来估计新的 $P1$ 和 $P2$ ，反复迭代下去，就可以最终得到 $P1 = 0.4$ ， $P2 = 0.5$ ，此时无论怎样迭代， $P1$ 和 $P2$ 的值都会保持0.4和0.5不变，于是就找到了 $P1$ 和 $P2$ 的最大似然估计。
- 这里有两个问题：
 - 1、新估计出的 $P1$ 和 $P2$ 一定会更接近真实的 $P1$ 和 $P2$ ？
答案是：没错，一定会更接近真实的 $P1$ 和 $P2$ ，**数学可以证明**。
 - 2、迭代一定会收敛到真实的 $P1$ 和 $P2$ 吗？
答案是：**不一定，取决于 $P1$ 和 $P2$ 的初始化值**，上面之所以能收敛到 $P1$ 和 $P2$ ，是因为幸运地找到了好的初始化值。

二、EM算法举例

□ 硬币A和B，H表示正面向上，T表示反面向上，参数 θ 表示正面朝上的概率。实验总共做了5组，每组随机选一个硬币，连续抛10次。

a Maximum likelihood



5 sets, 10 tosses per set

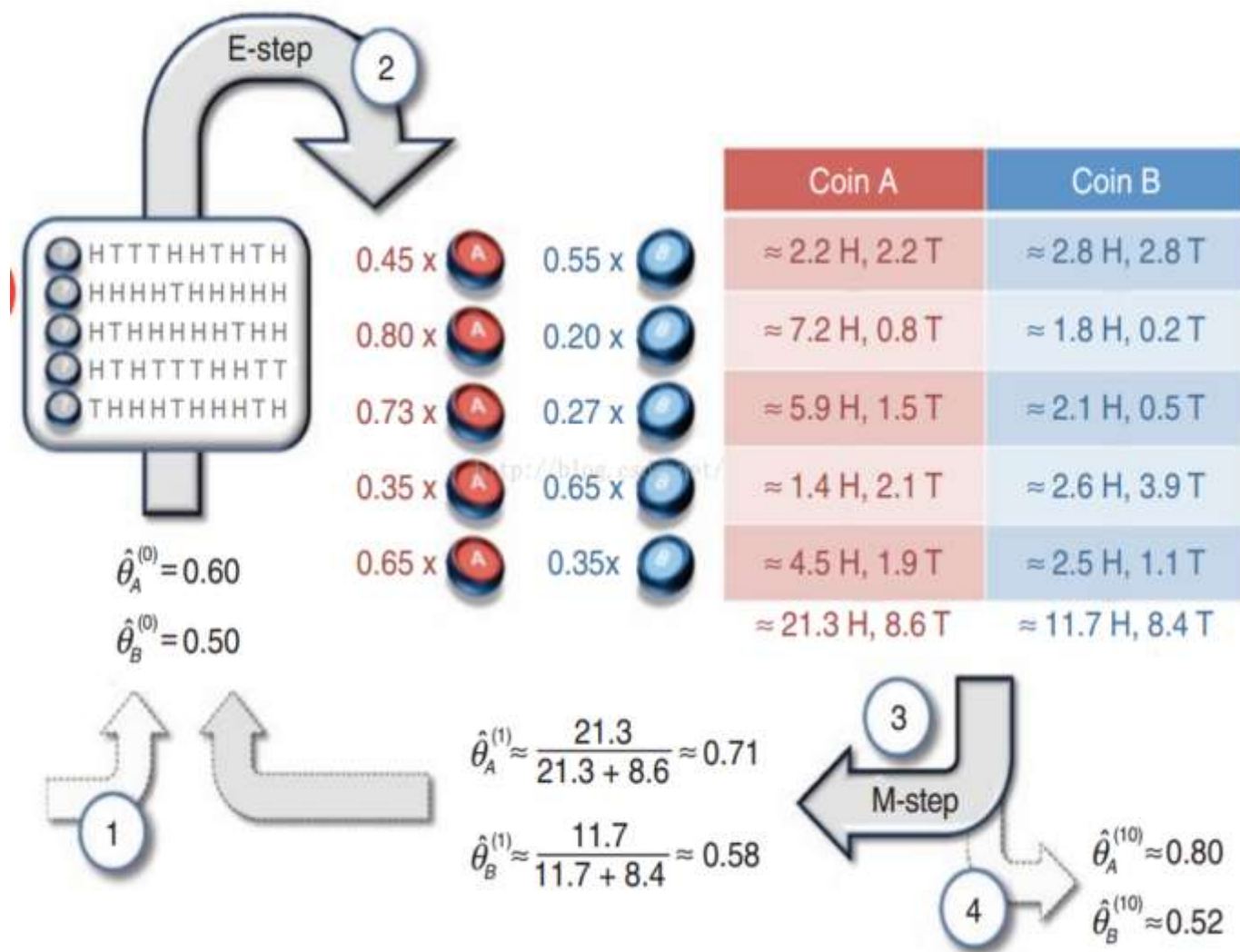
Coin A	Coin B
	5 H, 5 T
9 H, 1 T	
8 H, 2 T	
	4 H, 6 T
7 H, 3 T	
24 H, 6 T	9 H, 11 T

$$\hat{\theta}_A = \frac{24}{24 + 6} = 0.80$$

$$\hat{\theta}_B = \frac{9}{9 + 11} = 0.45$$

二、EM算法举例

b Expectation maximization



初始值 $\theta_A = 0.6$, $\theta_B = 0.5$

$$pA = C(10,5) * (0.6^5) * (0.4^5)$$

$$pB = C(10,5) * (0.5^5) * (0.5^5)$$

$$pA / (pA + pB) = 0.449$$

$$0.449 * 5H = 2.2H$$

$$0.449 * 5T = 2.2T$$

□ 根据假设特征的条件概率分布不同分为：

Sklearn.naive_bayes.GaussianNB 高斯贝叶斯分类器

➤ Sklearn.naive_bayes.MultinomialNB 多项式贝叶斯分类器

➤ Sklearn.naive_bayes.BernoulliNB 伯努利贝叶斯分类器

□ 场景选择

➤ 一般来说，如果样本**特征是连续值**，使用GaussianNB会比较好。

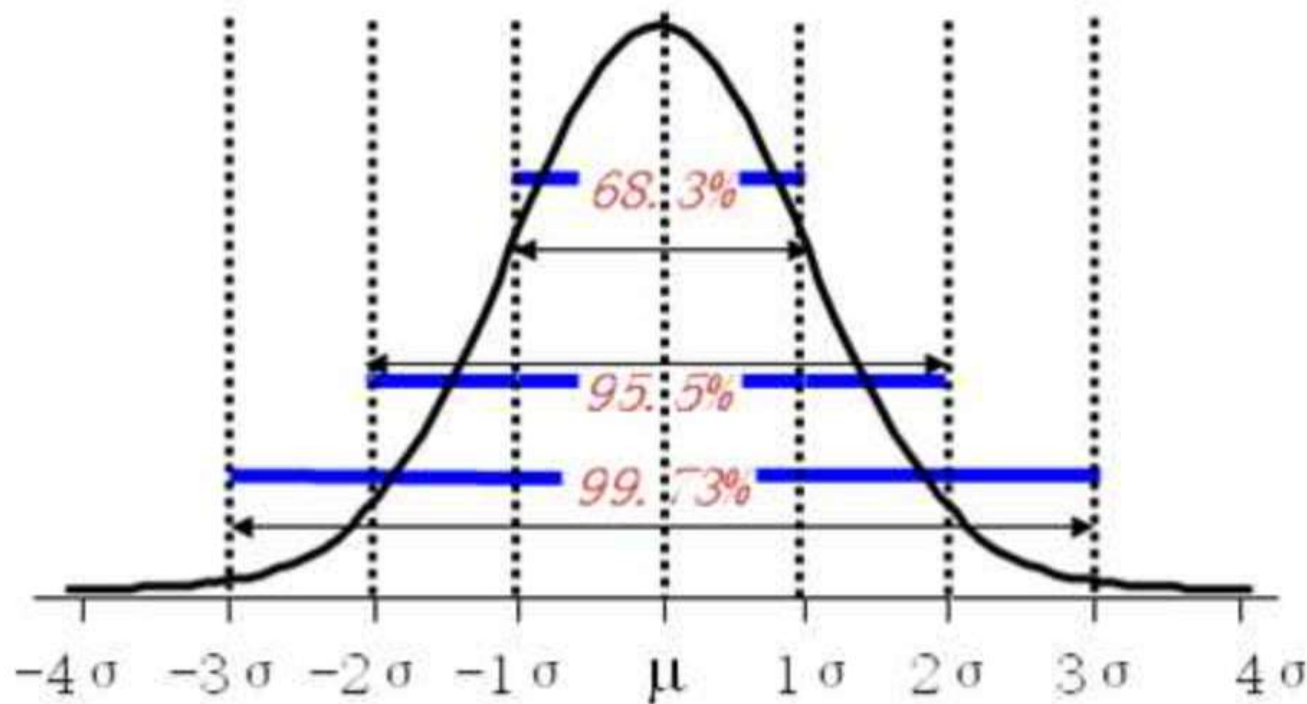
➤ 如果样本特征是**多元离散值**，使用MultinomialNB比较合适。

➤ 如果样本特征是二元离散值或者很稀疏的多元离散值，应该使用BernoulliNB。

四、高斯朴素贝叶斯

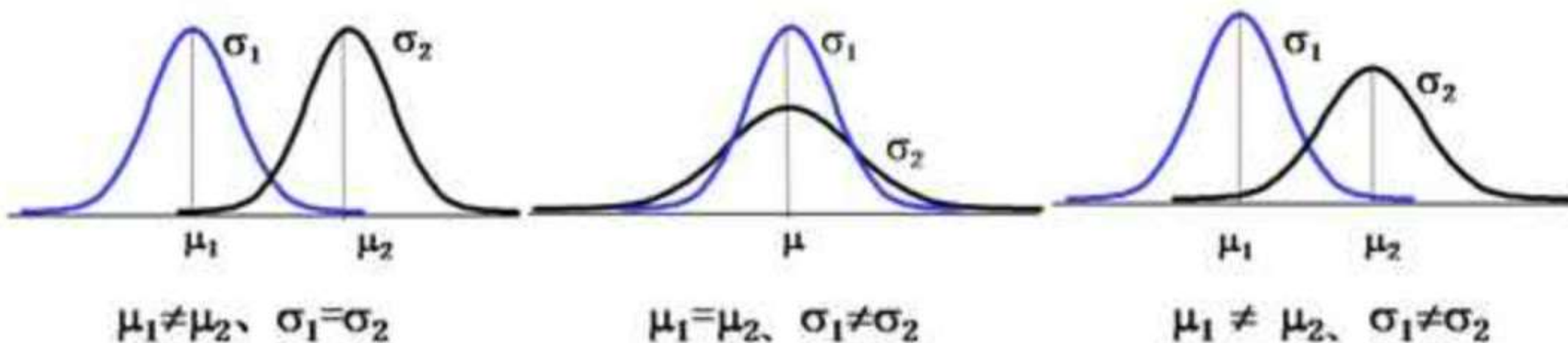
- GaussianNB: 有些特征是连续值, 比如人的身高, 可以离散化处理, 或者进行独热码编码, **但是都不够细腻**。高斯模型可以解决这个问题。
- 高斯分布也称为正态分布, 其条件概率函数可以写为:

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



四、高斯朴素贝叶斯

- 正态分布中均值决定曲线的位置，标准差决定曲线的胖瘦。



四、高斯朴素贝叶斯



□ 一组人类身体特征的统计资料。已知某人身高6英尺、体重130磅，脚掌8英寸。

□ 请问该人是男是女？

性别	身高 (英尺)	体重 (磅)	脚掌 (英寸)
男	6	180	12
男	5.92	190	11
男	5.58	170	12
男	5.92	165	10
女	5	100	6
女	5.5	150	8
女	5.42	130	7
女	5.75	150	9

四、高斯朴素贝叶斯

□ 由于身高、体重、脚掌都是连续变量，不能采用离散变量的方法计算概率。而且由于样本太少，所以也无法分成区间计算。怎么办？

□ 可以假设男性和女性的身高、体重、脚掌都是正态分布，通过样本计算出均值和方差，也就是得到正态分布的密度函数。有了密度函数，就可以把值代入，算出某一点的密度函数的值。比如，男性的身高是均值5.855、方差0.035的正态分布。所以，男性的身高为6英尺的概率的相对值等于1.5789（大于1并没有关系，因为这里是密度函数的值，只用来反映各个值的相对可能性）。

$$p(\text{height}|\text{male}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(\frac{-(6 - \mu)^2}{2\sigma^2}\right) \approx 1.5789$$

四、高斯朴素贝叶斯

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn import metrics

iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, \
                                                    test_size=0.25, random_state=1)

gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred = gnb.predict(X_test)

print("Gaussian Naive Bayes model accuracy(in %):", \
      metrics.accuracy_score(y_test, y_pred)*100)
```

四、多项式朴素贝叶斯

□ 该模型常用于文本分类，**特征是单词，值是单词的出现次数。**

```
>>> import numpy as np
>>> X = np.random.randint(5, size=(6, 100))
>>> y = np.array([1, 2, 3, 4, 5, 6])
>>> from sklearn.naive_bayes import MultinomialNB
>>> clf = MultinomialNB()
>>> clf.fit(X, y)
```

四、伯努利朴素贝叶斯

□ 与多项式模型一样，伯努利模型适用于离散特征的情况，所不同的是，伯努利模型中每个特征的取值只能是1和0(以文本分类为例，某个单词在文档中出现过，则其特征值为1，否则为0)。

伯努利模型中，条件概率 $P(x_i|y_k)$ 的计算方式是：

当特征值 x_i 为1时， $P(x_i|y_k) = P(x_i = 1|y_k)$;

当特征值 x_i 为0时， $P(x_i|y_k) = 1 - P(x_i = 1|y_k)$;

四、伯努利朴素贝叶斯

```
>>> import numpy as np
>>> X = np.random.randint(2, size=(6, 100))
>>> Y = np.array([1, 2, 3, 4, 4, 5])
>>> from sklearn.naive_bayes import BernoulliNB
>>> clf = BernoulliNB()
>>> clf.fit(X, Y)
```



重庆大学
CHONGQING UNIVERSITY

THANKS
祝学习快乐!